

نقشه ساخت Karyar Core

Karyar Core Roadmap

نسخه ۱/۰ — ۹ مهر ۱۴۰۵ / ۱ اکتبر ۲۰۲۶

مکمل «سند معماری Karyar نسخه ۱/۰». سرور، DevOps، سایت و UI عمومی در این سند نیست.

فهرست

- بخش ۱: معماری Core در یک تصویر
- بخش ۲: چه چیزی ثابت است و چه چیزی تغییر می‌کند
- بخش ۳: نگهداری Workflowها
- بخش ۴: نقش هر جزء
- بخش ۵: Workflow Builder
- بخش ۶: نقش GitHub و Claude Code
- بخش ۷: MVP – چه بسازیم و چه نسازیم
- بخش ۸: ساختار Pack برای هر مشتری
- بخش ۹: Roadmap مرحله به مرحله
- بخش ۱۰: شروع از همین هفته
- بخش ۱۱: تصمیم‌های لازم

۱. معماری Core در یک تصویر

Karyar در سه لایه ساخته می‌شود:

Customer Pack (YAML files in GitHub) agents, processes, approval rules, templates	← different per customer ← the only thing AI Builder edits
Karyar Core (one Frappe app: karyar) agent engine, Task, Approval, Permission, Promote, Idempotency, Pack loader, n8n generator	← same for everyone ← only developers (Claude Code) change it
ERPNext (business logic + business data)	← untouched

n8n ← runs the processes that Core generates from the Pack

قاعده طلایی: Core «موتور» است و Pack «تنظیمات».

اگر برای مشتری جدید مجبور شدید که Core را تغییر دهید، یکی از این دو حالت است:

- یک قابلیت عمومی کم بوده که باید به Core اضافه شود و همه مشتری‌ها از آن سود می‌برند.
- یا مسیر اشتباه است.

وضعیت فعلی: نمونه‌های فعلی (هانا، آوا، پنل‌ها، ثبت لید) کاملاً داخل n8n ساخته شده‌اند. منطق کسب‌وکار، دیتابیس Postgres با جدول‌های `karyar_*`، و مجوز با کد ورود همه در n8n است. این برای نمونه اولیه درست بوده، ولی همان‌گویی است که Core جایگزینش می‌کند. **فعلاً به آن‌ها دست نمی‌زنیم** و از آن‌ها فقط به‌عنوان نیازمندی‌های اولین مشتری استفاده می‌کنیم.

۲. چه چیزی ثابت است و چه چیزی تغییر می‌کند

ثابت (Core، کد)	قابل تغییر (Pack، فایل تنظیمات)
انواع گام فرایند: <code>promote</code> ، <code>approval</code> ، <code>human_task</code> ، <code>agent_task</code> ، <code>condition</code> ، <code>integration</code> ، <code>notify</code>	فرایندها: کدام گام‌ها، با چه ترتیبی، با چه شرطی
ابزارهای ایجنت (Tools)، مثل «ساخت Lead» و «خواندن گزارش»	کدام ابزارها به کدام ایجنت داده شوند
موتور ایجنت، اتصال به مدل AI، ثبت مصرف	ایجنت‌ها: نام، شخصیت، دستورالعمل، مدل، کانال
Approval، Human Task، Change Proposal، موتور	قوانین تأیید: چه کسی، چند نفر، با چه ترتیبی
Permission، Promote، Idempotency، Audit	نقش‌ها و انتساب افراد و ایجنت‌ها به نقش‌ها
بارگذار Pack و اعتبارسنج آن (Validator)	فیلدهای اضافه <code>Human</code> ، <code>Task</code> ، گزارش‌های مجاز، <code>Template</code> ، <code>ERPNext</code> ، <code>Human</code>
تولیدکننده Workflow برای n8n	متن پیام‌ها، سقف‌ها، و مدت نگهداری داده

۳. نگهداری Workflowها

۳.۱ مقایسه روش‌ها

روش	مشکل اگر به‌تنهایی استفاده شود
فقط کد (Python)	AI نمی‌تواند بی‌خطر تغییرش دهد. هر مشتری به برنامه‌نویس نیاز دارد
فقط دیتابیس (فرم‌ها)	بدون نسخه، تاریخچه، بازبینی و امکان بازگشت
فقط JSON n8n	بزرگ و شکننده. AI در ویرایش آن خطا می‌کند و منطق پراکنده می‌شود
✓ فایل YAML در Git، به‌عنوان منبع حقیقت	خوانا برای انسان و AI، نسخه‌دار، قابل بازبینی و قابل اعتبارسنجی

۳.۲ مسیر

YAML in GitHub (source of truth)

- Validator checks it
- apply-pack loads it into the Karyar database (runtime copy)
- generator builds and publishes the n8n workflows from it

۳.۳ نمونه یک فرایند در Pack

```
process: clinic_intake
version: 3
trigger: intake.completed
steps:
  - id: review
    type: human_task
    template: staff_review
    assignee: responsibility:reception
  - id: register
    type: promote
    operation: crm.lead
    approval: none
  - id: handoff
    type: agent_task
    responsibility: sales      # currently assigned to Hanna; the name never appears here
```

⚠️ **اصلاح پیشنهادی نسبت به سند معماری ۱/۱ (CR-06):** در سند فعلی آمده «Workflowها در ویرایشگر n8n ساخته می‌شوند». برای اینکه AI Builder بتواند Workflow بسازد، پیشنهاد می‌شود منبع حقیقت YAML باشد و n8n از روی آن تولید شود. n8n همچنان مجری است و این تغییر با بقیه معماری تناقض ندارد. نیاز به تأیید دارد.

۴. نقش هر جزء

نقش	جزء
ثبت، محاسبه و اعتبارسنجی کسب‌وکار. منبع حقیقت داده	ERPNext
موتور: ایجنت‌ها، Task، Approval، Permission، Promote، بارگذار Pack، تولیدکننده n8n	Karyar Core
اجرای فرایندهایی که Core ساخته، و اتصال به سرویس‌های بیرونی (پیامک، APIها)	n8n
داخل Core، با ابزارهای مجاز، روی داده واقعی کار می‌کنند	ایجنت‌های Runtime (آوا، هانا و...)
فقط Pack را تغییر می‌دهد. به کد Core و داده مشتری دسترسی ندارد	Workflow Builder

۵. Workflow Builder چطور کار کند

Builder مثل یک کارمند تازه‌وارد دقیق است که فقط حق پیشنهاد دارد:

1. شما توضیح می‌دهید: «فرایند فروش این مشتری این‌طور باشد...»
2. Builder پیشنهاد می‌دهد: فقط فایل‌های YAML همان Pack را تغییر می‌دهد و نتیجه را به شکل یک Pull Request با توضیح فارسی تحویل می‌دهد.
3. بررسی خودکار (Validator) پیش از اینکه شما چیزی ببینید:
 - هیچ ابزار، نقش یا Template ناموجودی به کار نرفته باشد.
 - هر promote قانون تأیید یا Policy اجرای خودکار داشته باشد.
 - هیچ ایجنتی بیش از اختیارش ابزار نگرفته باشد.
4. شبیه‌سازی: چند سناریوی آزمایشی (مثلاً «مشتری اطلاعاتش را کامل نداد») روی محیط تست اجرا می‌شود.

5. شما تأیید می‌کنید (Merge) و Pack روی آن مشتری اعمال می‌شود.
6. اگر قابلیت در Core نباشد، Builder آن را خودش نمی‌سازد. یک Issue باز می‌کند: «برای این کار به ابزار X نیاز است».
- نسخه اول Builder همان Claude Code است، همراه با یک دستورالعمل اختصاصی (Skill) برای ویرایش Pack.
- Builder درون محصول (با رابط گفت‌وگو) مرحله بعد است.

۶. نقش Claude Code و GitHub

نقش	
GitHub	منبع حقیقت همه‌چیز. مخزن karyar (اپ Core) و مخزن karyar-packs (برای هر مشتری یک پوشه). هر تغییر از طریق PR انجام می‌شود و تست‌ها خودکار اجرا می‌شوند
Claude Code	(۱) سازنده Core: شما مشخصات را به فارسی می‌نویسید و Claude Code کد و تست را می‌نویسد و PR می‌دهد. (۲) نسخه اول Workflow Builder
شما (معمار)	نوشتن مشخصات در قالب Issue، بررسی توضیح PR و نتیجه تست‌ها، و Merge. خواندن کد لازم نیست
CLAUDE.md	مهم‌ترین فایل پروژه. قوانین معماری نسخه ۱/۱ به صورت دستور در آن نوشته می‌شود (مثلاً «هرگز SQL ننویس»، «هر promote باید Idempotency داشته باشد»). Claude Code هر بار آن را می‌خواند

چرخه کاری شما در هر مرحله:

1. یک Issue با توضیح فارسی بنویسید: «چه می‌خواهم» و «چطور بفهمم درست کار می‌کند».
2. از Claude Code بخواهید آن Issue را پیاده کند و PR بدهد.
3. توضیح PR و نتیجه تست‌ها را بخوانید. اگر چیزی مبهم بود، از خود Claude Code بپرسید.
4. اگر «علامت تمام شدن» آن مرحله برقرار بود، Merge کنید.

۷. MVP: چه بسازیم و چه نسازیم

می‌سازیم:

- فرمت Pack و Validator آن، به همراه یک Pack نمونه
- اپ karyar با DocType‌های اصلی: Agent، Case، Human Task، Approval، Audit، Idempotency
- موتور ایجنت با ۳ تا ۵ ابزار و یک مدل AI
- Approval (تک‌نفره و M از N)، Human Task، Change Proposal
- Promote برای Lead و Sales Order، با Idempotency
- انتشار رویداد و تولیدکننده n8n برای ۴ نوع گام: notify، promote، agent_task، human_task
- صفحه ساده گفت‌وگو و Task (حتی داخل Desk خود Frappe)

نمی‌سازیم:

- UI زیبا
- Builder گرافیکی
- چند کانال همزمان (فقط یک کانال)

- RPA، سامانه مودیان، حقوق و دستمزد
- صورت حساب دهی و Control Plane
- صوت و MCP
- انتقال سیستم های فعلی n8n

۸. ساختار Pack برای هر مشتری

```
karyar-packs/
├── _base/                shared by everyone (base roles, generic templates)
├── clinic/              industry pack: clinics
├── hana-badahang/       this customer's pack (only its differences from _base)
│   ├── pack.yaml       name, version, which pack it inherits from
│   ├── agents/          processes/ approvals/ templates/ erp_fields/
│   └── tests/           test scenarios for this customer
```

- مشتری جدید یعنی یک پوشه جدید. از `_base` یا Pack صنفی ارث می برد و فقط تفاوت هایش را اضافه می کند.
- هانا بداهنگ اولین مشتری روی Core است. Workflow های فعلی او در n8n فقط مرجع نیازمندی ها هستند. نسخه جدید روی Core ساخته می شود و نسخه فعلی تا آماده شدن آن کار می کند.
- دوباره کاری نمی شود: منطق یک بار درست ساخته می شود و برای مشتری بعدی هم قابل استفاده است.

۹. Roadmap مرحله به مرحله

هر مرحله سه بخش دارد: شما چه می کنید، به Claude Code چه می گوئید، و از کجا می فهمید تمام شده (بدون خواندن کد).

#	مرحله	کار شما	به Claude Code	علامت تمام شدن
۰	پایه GitHub (۱ تا ۲ روز)	ساخت مخزن های karyar- و karyar packs	«یک CLAUDE.md بر اساس قوانین سند معماری ۱/۱ بنویس»	فایل را می خوانید و با قوانین موافقت
۱	فرمت Pack (۱ هفته)	نوشتن توضیح فارسی یک فرایند واقعی (مثلاً پذیرش لید هانا)	«فرمت Pack، Validator YAML و Pack، نمونه را بساز»	Pack درست قبول می شود و Pack عمداً خراب با پیام فارسی رد می شود
۲	اسکلت اپ (۱ هفته)	—	«اپ Frappe به نام karyar روی ERPNext v16 با DocType های اصلی و دستور apply-pack»	Pack نمونه بارگذاری می شود و ایجنت ها و فرایندها در Desk دیده می شوند
۳	مجوز و Audit (۱ هفته)	نوشتن ۵ سناریوی «چه کسی چه چیزی را نباید ببیند»	«زنجیره مجوز و Audit Log، با تست برای همین سناریوها»	همه تست های مجوز سبز
۴	ایجنت اول (۱ تا ۲ هفته)	تعیین ۳ تا ۵ ابزار اول	«موتور ایجنت، اتصال مدل، ثبت مصرف، و این ابزارها»	با ایجنت گفت و گو می کنید و او Lead پیش نویس می سازد
۵	Human Task و Approval (۲ هفته)	تعیین قوانین تأیید برای ۲ عملیات	«Human Task، Change Proposal، و Approval (تک نفره و M از N)»	تغییر سن با Diff نمایش داده می شود و تا تأیید نشود ثبت نمی شود

#	مرحله	کار شما	به Claude Code	علامت تمام شدن
۶	Promote و Idempotency ۱) (هفته)	—	«Promote برای Lead و Sales Order از طریق ERPNext API، با Idempotency»	دوبار زدن دکمه فقط یک سند می‌سازد
۷	اتصال n8n ۱) تا ۲ (هفته)	—	«انتشار رویداد و تولیدکننده n8n برای ۴ نوع گام»	فرایند YAML خودکار از پذیرش تا ارجاع به هانا جلو می‌رود
۸	Pack هانا ۱) تا ۲ (هفته)	توضیح کامل فرایندهای هانا	«hana-badahang Pack را بساز و سناریوهای تست را بنویس»	سناریوهای هانا روی Core اجرا می‌شوند
۹	Builder نسخه ۱ (هفته)	—	«یک Skill برای تغییر Pack با زبان فارسی، همراه با Validator و PR»	می‌گویید «یک مرحله تأیید مدیر اضافه کن» و یک PR درست و سبز تحویل می‌گیرید
۱۰	مشتری دوم	—	«clinic Pack را از روی _base بساز»	ثابت می‌شود Core بدون تغییر کد، مشتری دوم را می‌پذیرد

زمان‌ها تقریبی‌اند. **قاعده:** تا «علامت تمام شدن» یک مرحله برقرار نشده، سراغ مرحله بعد نروید.

۱۰. شروع از همین هفته

از «قرارداد» شروع کنید، نه از کد آپ.

- امروز:** در GitHub دو مخزن خالی بسازید: `karyar` و `karyar-packs`.
- روز اول با Claude Code:** «سند docs/karyar/KARYAR_ARCHITECTURE.fa.md را بخوان و یک CLAUDE.md برای مخزن karyar بنویس که همه قوانین آن را به صورت دستور شامل شود.» خروجی را بخوانید تا مطمئن شوید قوانین درست فهمیده شده‌اند.
- بقیه هفته: مرحله ۱ (فرمت Pack و Validator).** چرا اول این؟
 - به ERPNext و سرور نیاز ندارد، پس سریع و کم‌دردسر است.
 - همه مراحل بعدی (اپ، Builder، n8n) روی آن سوار می‌شوند.
 - کار اصلی شما در آن است:** توضیح فرایند به زبان ساده. Claude Code آن را به YAML و Validator تبدیل می‌کند.
- بعد** مراحل ۲ تا ۱۰ را به ترتیب جدول جلو ببرید. هر مرحله یک PR است.

۱۱. تصمیم‌های لازم

#	تصمیم	پیشنهاد
۱	CR-06: فایل YAML منبع حقیقت باشد و n8n از روی آن تولید شود	تأیید
۲	اولین Pack واقعی، هانا بداهنگ باشد	تأیید

این سند مکمل «سند معماری Karyar نسخه ۱/۱» است و همه قواعد آن (ERPNext منبع منطق و داده، Draft بدون نقش ERP دوم، n8n، Idempotency مشترک و Tenant-aware) برقرار است. سرور، DevOps، سایت و UI عمومی عمداً خارج از این سند است.